Aggregation Pipeline

Do que já vimos ao que se usa no dia a dia — com exemplos e um teste de contratação

Banco de Dados NoSQL • MongoDB • Sessão do Engenheiro(a) Sênior

Como Vai Ser Esta Sessão

GRADUAÇÃO
UniCesumar

1

Revisão rápida

$match, $group, $unwind, $lookup, $facet… o que já dominamos

2

Pipelines novos

datas, arrays, janelas, $lookup com sub-pipeline, $merge…

3

Desempenho e erros

boas práticas e as armadilhas mais comuns

4

Teste prático

10 questões • 10 pontos • mesma base de dados da aula

Regra do jogo: cada pipeline desta aula tem um irmão no teste. Prestem atenção na ordem dos estágios.

A Base de Dados: StreamTech

Plataforma de streaming fictícia. Três coleções: 12 títulos, 8 assinantes, 48 visualizações.

titulos

_id, titulo, tipo (filme|serie)

generos: [...]

ano, duracaoMin, notaImdb, pais

elenco: [{nome, papel}]

assinantes

_id, nome, email

plano: basico|padrao|premium

cidade, uf, ativo

dataCadastro (Date)

visualizacoes

_id, assinanteId → assinantes

tituloId → titulos

dataHora (Date)

minutosAssistidos, dispositivo

avaliacao (pode não existir!)

Carga: load("streamtech_seed.js") no mongosh — cria o banco streamtech.

Atenção: 45% das visualizações não têm o campo avaliacao — isso é de propósito (veremos $ifNull).

O Modelo Mental do Pipeline

GRADUAÇÃO
UniCesumar



- **A saída de cada estágio é a entrada do próximo. Mudou a ordem, mudou o resultado.**
- Filtre cedo: $match e $project no começo reduzem o que os estágios caros precisam processar.
- $project (inclusão) descarta tudo o que não for listado — cuidado com campos que serão usados depois (ex.: borough/assinanteId).
- Depois de um $group, o documento só tem _id + os acumuladores. O campo de agrupamento mora em _id.
- Para usar o VALOR de um campo dentro de uma expressão: "$campo". Sem o $, é texto literal.

Mapa de Estágios da Aula

GRADUAÇÃO
UniCesumar

REVISÃO — já trabalhamos

`$match` • `$project` • `$sort` • `$limit`

`$group` (+ `$sum` `$avg` `$max` `$push` `$addToSet`)

`$unwind` • `$addFields`

`$cond` • `$switch`

`$bucket` • `$bucketAuto`

`$lookup` + `$unwind`

`$facet`

NOVOS — dia a dia de trabalho

`$dateToString` (datas)

`$filter` • `$map` • `$reduce` • `$size` (arrays)

`$ifNull` • `$concat` • `$toUpper`

`$sortByCount`

`$replaceRoot` + `$mergeObjects`

`$setWindowFields` (`$rank`, acumulado)

`$unionWith`

`$lookup` com let/pipeline

`$expr` dentro do `$match`

`$merge` • `$out`

Revisão: Filtrar e Ordenar

GRADUAÇÃO
UniCesumar

REVISÃO

$match → $project → $sort → $limit: o esqueleto de quase toda consulta.

mongosh

```
db.visualizacoes.aggregate([
  { $match: { dispositivo: "tv", minutosAssistidos: { $gte: 100 } } },
  { $project: { assinanteId: 1, tituloId: 1, minutosAssistidos: 1 } },
  { $sort: { minutosAssistidos: -1, _id: 1 } },
  { $limit: 3 }
])
```

Resultado (base StreamTech)

```
{ _id: 7, assinanteId: 2, tituloId: 8, minutosAssistidos: 140 }
{ _id: 16, assinanteId: 3, tituloId: 5, minutosAssistidos: 110 }
```

- **Sempre desempate o $sort (ex.: _id: 1) quando houver valores repetidos — o resultado fica determinístico.**

Revisão: $group

GRADUAÇÃO
UniCesumar

REVISÃO

Agrupar e resumir: contagem, soma e média por dispositivo.

mongosh

```
db.visualizacoes.aggregate([
  { $group: {
      _id: "$dispositivo",
      total:   { $sum: 1 },
      minutos: { $sum: "$minutosAssistidos" },
      media:   { $avg: "$minutosAssistidos" } } },
  { $project: { total: 1, minutos: 1,
                media: { $round: ["$media", 1] } } },
  { $sort: { minutos: -1 } }
])
```

Resultado (base StreamTech)

```
{ _id: "web", total: 15, minutos: 1299, media: 86.6 }
{ _id: "tablet", total: 15, minutos: 1123, media: 74.9 }
{ _id: "tv", total: 12, minutos: 606, media: 50.5 }
{ _id: "celular", total: 6, minutos: 396, media: 66 }
```

- **O _id do $group é a chave de agrupamento. Cada acumulador cria um campo novo no resultado.**

Revisão: $unwind + $addFields

GRADUAÇÃO
UniCesumar

REVISÃO

Desmontar o array de gêneros e calcular a nota média de cada gênero.

mongosh

```
db.titulos.aggregate([
  { $unwind: "$generos" },
  { $group: { _id: "$generos",
              qtd: { $sum: 1 },
              notaMedia: { $avg: "$notaImdb" } } },
  { $addFields: { notaMedia: { $round: ["$notaMedia", 2] } } },
  { $sort: { qtd: -1, _id: 1 } }
])
```

Resultado (base StreamTech)

```
{ _id: "Drama", qtd: 4, notaMedia: 7.57 }
{ _id: "Ação", qtd: 3, notaMedia: 8.03 }
{ _id: "Comédia", qtd: 3, notaMedia: 6.7 }
{ _id: "Suspense", qtd: 3, notaMedia: 7.77 }
// … e mais 4 documento(s)
```

- **$unwind gera 1 documento por elemento do array. Documentos com array vazio somem.**

Revisão: $switch

Classificar cada visualização por engajamento com vários ramos.

mongosh

```
db.visualizacoes.aggregate([
  { $addFields: { engajamento: { $switch: {
      branches: [
        { case: { $gte: ["$minutosAssistidos", 120] }, then: "Maratona" },
        { case: { $gte: ["$minutosAssistidos", 45] },  then: "Normal" }
      ],
      default: "Abandono" } } } },
  { $group: { _id: "$engajamento", qtd: { $sum: 1 } } },
  { $sort: { qtd: -1 } }
])
```

Resultado (base StreamTech)

```
{ _id: "Normal", qtd: 21 }
{ _id: "Abandono", qtd: 14 }
{ _id: "Maratona", qtd: 13 }
```

- **Use $switch quando houver 3+ faixas; evita $cond aninhado. O default é obrigatório na prática.**

Revisão: $bucket

Distribuição por faixas de duração, com limites escolhidos por você.

mongosh

```
db.titulos.aggregate([
  { $bucket: {
      groupBy: "$duracaoMin",
      boundaries: [0, 60, 100, 130],
      default: "130+",
      output: { qtd: { $sum: 1 },
                maisLongo: { $max: "$duracaoMin" } } } }
])
```

Resultado (base StreamTech)

```
{ _id: 0, qtd: 4, maisLongo: 58 }
{ _id: 60, qtd: 4, maisLongo: 98 }
{ _id: 100, qtd: 2, maisLongo: 121 }
{ _id: "130+", qtd: 2, maisLongo: 142 }
```

- **Os boundaries definem [min, max). Valores fora das faixas vão para o default.**

Revisão: $bucketAuto

Deixar o MongoDB escolher faixas com quantidades equilibradas.

mongosh

```
db.titulos.aggregate([
  { $bucketAuto: { groupBy: "$notaImdb", buckets: 3,
                   output: { qtd: { $sum: 1 } } } }
])
```

Resultado (base StreamTech)

```
{ _id: { min: 6.2, max: 7.4 }, qtd: 4 }
{ _id: { min: 7.4, max: 8.3 }, qtd: 4 }
{ _id: { min: 8.3, max: 9 }, qtd: 4 }
```

- **Use $bucket quando você conhece as faixas; $bucketAuto quando quer quantis/equilíbrio.**

Revisão: $lookup

Juntar visualizações a títulos (o "JOIN" do MongoDB) e somar minutos por título.

mongosh

```
db.visualizacoes.aggregate([
  { $lookup: { from: "titulos", localField: "tituloId",
               foreignField: "_id", as: "t" } },
  { $unwind: "$t" },
  { $group: { _id: "$t.titulo",
              minutos: { $sum: "$minutosAssistidos" } } },
  { $sort: { minutos: -1, _id: 1 } },
  { $limit: 3 }
])
```

Resultado (base StreamTech)

```
{ _id: "Amor em Dados", minutos: 529 }
{ _id: "Comédia de Bar", minutos: 485 }
{ _id: "Reino de Gelo", minutos: 361 }
```

- **$lookup devolve um ARRAY (campo as). Quase sempre vem seguido de $unwind.**

Revisão: $facet

Vários resumos independentes sobre a mesma entrada, em uma única consulta.

- **Cada faceta é um mini-pipeline. Ideal para dashboards e telas com vários painéis.**

mongosh

```
db.titulos.aggregate([
  { $facet: {
      porTipo:  [ { $group: { _id: "$tipo", qtd: { $sum: 1 } } },
                  { $sort: { _id: 1 } } ],
      melhores: [ { $sort: { notaImdb: -1 } }, { $limit: 2 },
                  { $project: { _id: 0, titulo: 1, notaImdb: 1 } } ],
      resumo:   [ { $group: { _id: null, total: { $sum: 1 },
                              notaMedia: { $avg: "$notaImdb" } } },
                  { $project: { _id: 0, total: 1,
                      notaMedia: { $round: ["$notaMedia", 2] } } } ]
  } }
])
```

Resultado (base StreamTech)

```
{
  porTipo: [ { _id: "filme", qtd: 7 }, { _id: "serie", qtd: 5 } ],
  melhores: [ { titulo: "Planeta Azul 2", notaImdb: 9 }, { titulo: "Reino de Gelo", notaImdb: 8.9 } ],
  resumo: [ { total: 12, notaMedia: 7.74 } ]
}
```

Agora, os Pipelines Novos

Os que mais aparecem no código de produção — e no nosso teste

Conceito: Datas e $dateToString

GRADUAÇÃO

UniCesumar

NOVO

PARA QUE SERVE

Converte um campo Date em texto formatado (ano, mês, dia, hora…). É a base de qualquer relatório por período: o texto vira a chave do $group.

sintaxe

```
{ $dateToString: {
    format: "%Y-%m",        // %Y ano • %m mês • %d dia • %H hora
    date: "$campoData",
    timezone: "America/Sao_Paulo"   // opcional
} }
```

ONDE USAR NO DIA A DIA

- Faturamento ou pedidos por mês, por dia ou por hora do dia.
- Novos cadastros por mês (curva de crescimento de usuários).
- Acessos por dia da semana: use $dayOfWeek, $month, $year.
- Agrupar por semana ou trimestre com $dateTrunc (MongoDB 5.0+).

ATENÇÃO: Datas ficam em UTC. Sem timezone, um evento das 22h no Brasil pode cair no dia seguinte.

Datas: $dateToString

GRADUAÇÃO
UniCesumar

NOVO

Agrupar por mês transformando um Date em texto "AAAA-MM".

mongosh

```
db.visualizacoes.aggregate([
  { $group: {
      _id: { $dateToString: { format: "%Y-%m", date: "$dataHora" } },
      minutos: { $sum: "$minutosAssistidos" },
      views:   { $sum: 1 } } },
  { $sort: { _id: 1 } },
  { $limit: 4 }
])
```

Resultado (base StreamTech)

```
{ _id: "2026-01", minutos: 408, views: 6 }
{ _id: "2026-02", minutos: 452, views: 4 }
{ _id: "2026-03", minutos: 510, views: 6 }
{ _id: "2026-04", minutos: 584, views: 9 }
```

- **Formatos úteis: %Y-%m-%d, %Y-%m, %H. Outros: $year, $month, $dayOfWeek, $dateTrunc.**

Conceito: $expr no $match

GRADUAÇÃO
UniCesumar

NOVO

PARA QUE SERVE

Permite usar expressões de agregação dentro de um filtro. Com ele, comparamos um campo com outro campo do MESMO documento, algo impossível com o $match tradicional.

sintaxe

```
{ $match: { $expr: { $gte: [ "$campoA", "$campoB" ] } } }


// também aceita $and, $or, $cond, cálculos:
{ $match: { $expr: { $gt: [ { $multiply: ["$preco", "$qtd"] }, 1000 ] } } }
```

ONDE USAR NO DIA A DIA

- Estoque abaixo do estoque mínimo.
- Data de término maior que a data de início (validação).
- Gasto acima do limite do plano.
- Minutos assistidos maiores ou iguais à duração do título.

ATENÇÃO: Em operadores de comparação o $expr pode não aproveitar índices tão bem quanto um filtro simples: use também um $match comum antes, quando possível.

Comparar Campos: $expr

GRADUAÇÃO

UniCesumar

NOVO

$match comum compara campo com valor fixo; $expr compara campo com campo.

mongosh

```
db.visualizacoes.aggregate([
  { $lookup: { from: "titulos", localField: "tituloId",
               foreignField: "_id", as: "t" } },
  { $unwind: "$t" },
  // compara DOIS campos do mesmo documento:
  { $match: { $expr: { $gte: ["$minutosAssistidos", "$t.duracaoMin"] } } },
  { $project: { titulo: "$t.titulo", minutosAssistidos: 1,
                duracao: "$t.duracaoMin" } },
  { $sort: { _id: 1 } },
  { $limit: 4 }
])
```

Resultado (base StreamTech)

```
{ _id: 3, titulo: "Planeta Azul 2", minutosAssistidos: 125, duracao: 49 }
{ _id: 5, titulo: "Os Últimos Bytes", minutosAssistidos: 62, duracao: 52 }
{ _id: 6, titulo: "Amor em Dados", minutosAssistidos: 125, duracao: 104 }
{ _id: 7, titulo: "Amor em Dados", minutosAssistidos: 140, duracao: 104 }
```

- **Aqui: visualizações em que os minutos assistidos cobriram a duração completa do título.**

Conceito: $ifNull, $concat e $toString

GRADUAÇÃO
UniCesumar

NOVO

PARA QUE SERVE

$ifNull devolve um valor padrão quando o campo está ausente ou null. Evita que o null contamine contas e textos. $toString e $toUpper formatam valores para relatórios.

sintaxe

```
{ $ifNull: [ "$campo", valorPadrao ] }
{ $toString: "$numero" }          { $toUpper: "$texto" }
{ $concat: [ "Nota: ", { $toString: { $ifNull: ["$nota", 0] } } ] }
```

ONDE USAR NO DIA A DIA

- Exibir "sem avaliação" quando a nota não existe.
- Telefone ou e-mail "não informado" em exportações.
- Contar ausente como zero antes de um $sum ou $avg.
- Padronizar textos (maiúsculas) e montar mensagens para o front-end.

ATENÇÃO: $ifNull só cobre ausente/null — não cobre string vazia. $avg ignora ausentes; se quiser contá-los como 0, aplique $ifNull antes.

Nulos e Texto: $ifNull

GRADUAÇÃO

UniCesumar

NOVO

Tratar campo ausente e formatar texto para relatórios.

mongosh

```
db.visualizacoes.aggregate([
  { $project: {
      dispositivo: { $toUpper: "$dispositivo" },
      nota:   { $ifNull: ["$avaliacao", "sem nota"] },
      resumo: { $concat: ["Assinante ", { $toString: "$assinanteId" },
                          " — ", { $toString: "$minutosAssistidos" }, " min"] } } },
  { $limit: 4 }
])
```

Resultado (base StreamTech)

```
{ _id: 1, dispositivo: "WEB", nota: "sem nota", resumo: "Assinante 6 — 74 min" }
{ _id: 2, dispositivo: "WEB", nota: "sem nota", resumo: "Assinante 2 — 47 min" }
{ _id: 3, dispositivo: "CELULAR", nota: 5, resumo: "Assinante 3 — 125 min" }
{ _id: 4, dispositivo: "CELULAR", nota: 5, resumo: "Assinante 2 — 125 min" }
```

- **$ifNull cobre campo ausente OU null. $concat só aceita strings: use $toString nos números.**

Conceito: $sortByCount

GRADUAÇÃO

UniCesumar

NOVO

PARA QUE SERVE

Atalho que conta quantos documentos existem para cada valor de um campo e ordena do mais frequente para o menos. Substitui $group + $sum + $sort.

sintaxe

```
{ $sortByCount: "$campo" }


// equivale a:
{ $group: { _id: "$campo", count: { $sum: 1 } } }
{ $sort:  { count: -1 } }
```

ONDE USAR NO DIA A DIA

- Produtos mais vendidos ou páginas mais acessadas.
- Cidades com mais clientes.
- Tipos de erro mais frequentes em logs.
- Tags mais usadas: $unwind do array e depois $sortByCount.

ATENÇÃO: O campo de saída se chama count (não total). Em empates a ordem não é garantida: use $group + $sort se precisar de desempate.

Atalho: $sortByCount

GRADUAÇÃO
UniCesumar

NOVO

Contar por categoria e ordenar do maior para o menor, em um estágio.

mongosh

```
db.titulos.aggregate([
  { $sortByCount: "$tipo" }   // = $group + { $sum: 1 } + $sort -1
])
```

Resultado (base StreamTech)

```
{ _id: "filme", count: 7 }
{ _id: "serie", count: 5 }
```

- **O campo do resultado chama-se count (não total). É o mesmo que $group + $sum + $sort -1.**

Conceito: $unionWith

GRADUAÇÃO
UniCesumar

NOVO

PARA QUE SERVE

Empilha, no resultado do pipeline, documentos de outra coleção. É o UNION ALL do SQL: junta dois conjuntos de linhas em um só.

sintaxe

```
{ $unionWith: {
    coll: "outraColecao",
    pipeline: [ { $project: { nome: 1, origem: "outra" } } ]   // opcional
} }
```

ONDE USAR NO DIA A DIA

- Vendas de 2025 e 2026 guardadas em coleções separadas.
- Clientes pessoa física e pessoa jurídica em um relatório único.
- Logs de várias origens ou serviços.
- Dados atuais mais histórico arquivado.

ATENÇÃO: Não remove duplicados. Alinhe os nomes dos campos com $project antes ou dentro do pipeline do $unionWith.

Unindo: $unionWith

GRADUAÇÃO
UniCesumar

NOVO

Empilhar documentos de duas coleções num único resultado.

mongosh

```
db.titulos.aggregate([
  { $match: { ano: { $gte: 2023 } } },
  { $project: { _id: 0, nome: "$titulo", origem: { $literal: "titulo" } } },
  { $unionWith: { coll: "assinantes", pipeline: [
      { $match: { plano: "premium" } },
      { $project: { _id: 0, nome: 1,
                    origem: { $literal: "assinante premium" } } } ] } }
])
```

Resultado (base StreamTech)

```
{ nome: "Cozinha Secreta", origem: "titulo" }
{ nome: "Velocidade Final", origem: "titulo" }
{ nome: "Guerra dos Reinos", origem: "titulo" }
{ nome: "Marina Alves", origem: "assinante premium" }
// … e mais 2 documento(s)
```

- **Equivale ao UNION ALL do SQL. Os esquemas precisam ser alinhados com $project.**

Conceito: $merge e $out

GRADUAÇÃO
UniCesumar

NOVO

PARA QUE SERVE

Gravam o resultado do pipeline em uma coleção. Transformam consultas pesadas em relatórios pré-calculados (views materializadas) e viabilizam ETL dentro do MongoDB.

sintaxe

```
{ $merge: { into: "colecao", on: "_id",
    whenMatched: "replace",      // replace | merge | keepExisting | fail
    whenNotMatched: "insert" } }
{ $out: "colecao" }          // apaga e recria a coleção inteira
```

ONDE USAR NO DIA A DIA

- Relatório diário gravado por um job noturno.
- Ranking pré-calculado para o dashboard ler rápido.
- ETL entre coleções ou bancos (staging → produção).
- Snapshots mensais de métricas.

ATENÇÃO: São sempre o ÚLTIMO estágio. $merge exige índice único no campo 'on'. $out substitui tudo — cuidado em produção.

Materializar: $merge e $out

GRADUAÇÃO
UniCesumar

NOVO

Gravar o resultado do pipeline em uma coleção (relatórios pré-calculados).

mongosh

```
db.visualizacoes.aggregate([
  { $group: { _id: "$dispositivo",
              minutos: { $sum: "$minutosAssistidos" } } },
  { $merge: { into: "relatorio_dispositivo",
              on: "_id",
              whenMatched: "replace",
              whenNotMatched: "insert" } }
])
// $out: { $out: "relatorio_dispositivo" }  → substitui a coleção inteira
```

- **$out substitui a coleção inteira; $merge faz upsert linha a linha. Sempre o ÚLTIMO estágio.**

Conceito: $filter, $map e $size

GRADUAÇÃO
UniCesumar

NOVO

PARA QUE SERVE

Operam DENTRO de um array do documento, sem $unwind. $filter escolhe elementos, $map transforma cada elemento e $size conta. O documento continua sendo um só.

sintaxe

```
{ $filter: { input: "$array", as: "x", cond: { $eq: ["$$x.campo", valor] } } }
{ $map:    { input: "$array", as: "x", in: "$$x.campo" } }
{ $size: "$array" }
```

ONDE USAR NO DIA A DIA

- Itens de um pedido com quantidade maior que 1.
- Somente os nomes do elenco, ou só quem tem papel principal.
- Quantidade de comentários, tags ou itens de cada documento.
- Produtos em promoção dentro do carrinho de compras.

ATENÇÃO: $size falha se o campo não for array: use $ifNull: ["$campo", []]. A variável é $$x (dois $); o campo é $campo (um $).

Arrays: $filter, $map, $size

GRADUAÇÃO

UniCesumar

NOVO

Trabalhar dentro do array, sem $unwind e sem multiplicar documentos.

mongosh

```
db.titulos.aggregate([
  { $match: { "elenco.0": { $exists: true } } },
  { $project: { _id: 0, titulo: 1,
      principais: { $map: {
          input: { $filter: { input: "$elenco", as: "e",
                              cond: { $eq: ["$$e.papel", "principal"] } } },
          as: "p", in: "$$p.nome" } },
      totalElenco: { $size: "$elenco" } } },
  { $limit: 3 }
])
```

Resultado (base StreamTech)

```
{ titulo: "Matrix Code", principais: ["Ana Prado", "Leo Brandt"], totalElenco: 3 }
{ titulo: "Noite em Curitiba", principais: ["Caio Ramos"], totalElenco: 2 }
{ titulo: "Os Últimos Bytes", principais: ["Ana Prado", "Rui Matos", "Tina Cruz"], totalElenco: 3 }
```

- **$filter escolhe elementos • $map transforma cada um • $size conta. $$e é a variável do elemento.**

Conceito: $reduce e $concat

GRADUAÇÃO
UniCesumar

NOVO

PARA QUE SERVE

$reduce percorre um array acumulando um único valor (soma, texto, maior valor). $concat junta textos. Juntos, montam strings e totais calculados a partir de listas.

sintaxe

```
{ $reduce: { input: "$array", initialValue: "",
    in: { $concat: ["$$value", "$$this.campo", ", "] } } }
{ $concat: [ "texto", "$campo", "outro texto" ] }   // só strings
```

ONDE USAR NO DIA A DIA

- Total do carrinho: somar preco x quantidade de cada item.
- Lista de tags ou de atores em uma única string: "Ana, Leo, Iris".
- Montar nome completo ou endereço a partir de partes.
- Maior ou menor valor de um array de objetos.

ATENÇÃO: $concat devolve null se qualquer parte for null. Para somar um array de números, $sum direto é mais simples.

Arrays: $reduce + $concat

GRADUAÇÃO
UniCesumar

NOVO

Reduzir um array a um único valor — aqui, um texto com os nomes do elenco.

mongosh

```
db.titulos.aggregate([
  { $match: { _id: { $in: [1, 3] } } },
  { $project: { _id: 0, titulo: 1,
      atores: { $reduce: {
          input: "$elenco", initialValue: "",
          in: { $concat: [ "$$value",
                  { $cond: [ { $eq: ["$$value", ""] }, "", ", " ] },
                  "$$this.nome" ] } } } } }
])
```

Resultado (base StreamTech)

```
{ titulo: "Matrix Code", atores: "Ana Prado, Leo Brandt, Iris Vale" }
{ titulo: "Os Últimos Bytes", atores: "Ana Prado, Rui Matos, Tina Cruz" }
```

- **$$value acumula, $$this é o elemento atual. Serve para somar, concatenar, achatar…**

Conceito: $replaceRoot e $mergeObjects

GRADUAÇÃO
UniCesumar

NOVO

PARA QUE SERVE

Troca o documento inteiro por um subdocumento (ou por um objeto calculado). Com $mergeObjects, funde objetos em um só. Deixa o resultado plano, sem aninhamento.

sintaxe

```
{ $replaceRoot: { newRoot: <expressão que resulta em objeto> } }
{ $replaceWith: <expressão> }                    // forma curta
{ $mergeObjects: [ <objeto1>, <objeto2> ] }
```

ONDE USAR NO DIA A DIA

- Achatar o resultado de um $lookup (juntar campos do pai e do filho).
- Promover um subdocumento (ex.: endereco) para a raiz.
- Entregar JSON plano para o front-end ou para exportação em CSV.
- Combinar campos originais com campos calculados em um único objeto.

ATENÇÃO: newRoot precisa ser um objeto. No $mergeObjects, se a chave se repete, o último objeto da lista vence.

Achatar: $replaceRoot

GRADUAÇÃO
UniCesumar

NOVO

Subir um objeto para a raiz do documento, combinando com $mergeObjects.

mongosh

```
db.visualizacoes.aggregate([
  { $match: { _id: { $lte: 3 } } },
  { $lookup: { from: "assinantes", localField: "assinanteId",
               foreignField: "_id", as: "a" } },
  { $replaceRoot: { newRoot: { $mergeObjects: [
        { $arrayElemAt: ["$a", 0] },
        { minutos: "$minutosAssistidos",
          dispositivo: "$dispositivo" } ] } } },
  { $project: { _id: 0, nome: 1, plano: 1, minutos: 1, dispositivo: 1 } }
])
```

Resultado (base StreamTech)

```
{ nome: "Diego Matos", plano: "basico", minutos: 74, dispositivo: "web" }
{ nome: "Paulo Nunes", plano: "basico", minutos: 47, dispositivo: "web" }
{ nome: "Joana Freitas", plano: "padrao", minutos: 125, dispositivo: "celular" }
```

- **Depois de um $lookup, $replaceRoot + $mergeObjects evita carregar o array “a” e deixa o documento plano.**

Conceito: $setWindowFields

GRADUAÇÃO

UniCesumar

NOVO

PARA QUE SERVE

Calcula valores sobre uma 'janela' de documentos vizinhos sem colapsar as linhas (diferente do $group). Faz ranking, acumulado, média móvel e comparação com a linha anterior.

sintaxe

```
{ $setWindowFields: {
    partitionBy: "$campo",              // opcional: reinicia por grupo
    sortBy: { valor: -1 },
    output: { novo: { $rank: {} },      // ou $sum, $avg, $shift…
              acum: { $sum: "$valor", window: { documents: ["unbounded","current"] } } }
} }
```

ONDE USAR NO DIA A DIA

- Ranking de vendedores por região (partitionBy).
- Faturamento acumulado mês a mês.
- Média móvel de 7 dias de acessos.
- Diferença para o mês anterior com $shift.

ATENÇÃO: Requer MongoDB 5.0+. $rank repete posição em empates ($denseRank não pula números). Defina sempre um sortBy determinístico.

Janelas: $setWindowFields

GRADUAÇÃO
UniCesumar

NOVO

Ranking e total acumulado sem perder as linhas (diferente do $group).

mongosh

```
db.visualizacoes.aggregate([
  { $group: { _id: "$tituloId",
              minutos: { $sum: "$minutosAssistidos" } } },
  { $setWindowFields: {
      sortBy: { minutos: -1, _id: 1 },
      output: {
        posicao:    { $rank: {} },
        acumulado: { $sum: "$minutos",
                     window: { documents: ["unbounded", "current"] } } } } },
  { $limit: 4 }
])
```

Resultado (base StreamTech)

```
{ _id: 8, minutos: 529, posicao: 1, acumulado: 529 }
{ _id: 4, minutos: 485, posicao: 2, acumulado: 1014 }
{ _id: 5, minutos: 361, posicao: 3, acumulado: 1375 }
{ _id: 12, minutos: 337, posicao: 4, acumulado: 1712 }
```

- **$rank dá empate na mesma posição; $denseRank não pula números; $documentNumber numera. Requer MongoDB 5.0+.**

Conceito: $lookup com sub-pipeline

GRADUAÇÃO

UniCesumar

NOVO

PARA QUE SERVE

Join correlacionado: para cada documento, roda um pipeline na outra coleção. Permite filtrar, ordenar, limitar e agregar ANTES de juntar — o que o $lookup simples não faz.

sintaxe

```
{ $lookup: {
    from: "outra",
    let: { v: "$campoLocal" },                    // variáveis locais
    pipeline: [ { $match: { $expr: { $eq: ["$campoRemoto", "$$v"] } } },
                { $sort: {...} }, { $limit: 3 } ],      // estágios extras
    as: "resultado" } }
```

ONDE USAR NO DIA A DIA

- Últimos 3 pedidos de cada cliente.
- Total gasto por cliente nos últimos 30 dias.
- Join por mais de um campo ou com condição (só ativos).
- Dados agregados da outra coleção (soma, média) sem $unwind + $group.

ATENÇÃO: Variáveis do documento local só entram via let e $$var. O resultado continua sendo um array. Indexe o campo remoto.

$lookup com Sub-pipeline

GRADUAÇÃO

UniCesumar

NOVO

Join correlacionado: filtra e agrega os documentos da outra coleção ANTES de juntar.

- **let cria variáveis ($$aid) usadas no $expr do sub-pipeline. Substitui "lookup + unwind + group".**

mongosh

```
db.assinantes.aggregate([
  { $match: { plano: "premium" } },
  { $lookup: {
      from: "visualizacoes",
      let: { aid: "$_id" },
      pipeline: [
        { $match: { $expr: { $and: [
            { $eq:  ["$assinanteId", "$$aid"] },
            { $gte: ["$dataHora", ISODate("2026-06-01")] } ] } } },
        { $group: { _id: null,
                    minutos: { $sum: "$minutosAssistidos" } } } ],
      as: "recentes" } },
  { $project: { _id: 0, nome: 1,
      minutosRecentes: { $ifNull: [ { $arrayElemAt: ["$recentes.minutos", 0] }, 0 ] } } }
])
```

Resultado (base StreamTech)

```
{ nome: "Marina Alves", minutosRecentes: 121 }
{ nome: "Rafael Costa", minutosRecentes: 207 }
{ nome: "Camila Rocha", minutosRecentes: 222 }
```

Desempenho e Boas Práticas

GRADUAÇÃO
UniCesumar

- $match e $sort no início, nos campos indexados, para o MongoDB usar índices.
- $project/$unset cedo para carregar menos campos; $limit assim que possível.
- Evite $unwind desnecessário: $filter, $map, $size e $reduce resolvem sem multiplicar documentos.
- $lookup: o foreignField deve ter índice. Prefira embedding quando o dado é lido sempre junto.
- db.colecao.explain("executionStats").aggregate([...]) mostra índice usado, docs examinados e tempo.
- Cada estágio tem limite de 100 MB de RAM: use { allowDiskUse: true } em agrupamentos enormes.
- Resultados de $facet e $group precisam caber em 16 MB por documento.
- Pipelines pesados e recorrentes? Materialize com $merge/$out e leia a coleção pronta.

Erros Clássicos (e a Cura)

GRADUAÇÃO
UniCesumar

ERRADO	CERTO	POR QUÊ
`street: "address.street"` (sem $)	`street: "$address.street"`	Sem o $ vira texto literal.
$project sem o campo usado depois	inclua borough: 1 antes do $group	Senão o $group agrupa tudo em _id: null.
$limit antes do $sort	$sort → $limit	Limitar primeiro pega docs quaisquer.
$group por "$borough" após outro $group	"$_id.borough"	Após o $group, o campo vive em _id.
pattern: "[0-100]" para faixa numérica	minimum: 0, maximum: 100	pattern é para strings (regex).
"grade" vs "grades" (singular/plural)	conferir o nome do campo	Campo inexistente = nenhum resultado.
$eq: ["classificacao", ...] (sem $)	$eq: ["$classificacao", ...]	Texto literal nunca é igual ao valor.

O Teste de Contratação

GRADUAÇÃO
UniCesumar

10 questões práticas em mongosh • 10 pontos • 90 minutos • base streamtech

Q1	`$match $sort $limit $project`	**0,50 pt**	**Q6**	`$bucket`	**1,00 pt**
Q2	`$group + acumuladores`	**0,75 pt**	**Q7**	`$dateToString`	**1,00 pt**
Q3	`$unwind $group top-N`	**1,00 pt**	**Q8**	`$filter $map $size`	**1,00 pt**
Q4	`$lookup $addToSet $size`	**1,25 pt**	**Q9**	`$facet`	**1,50 pt**
Q5	`$switch`	**1,00 pt**	**Q10**	`$setWindowFields`	**1,00 pt**

- Pode consultar a documentação oficial do MongoDB. Sem ajuda de colegas ou de IA.
- Entrega: um arquivo .js (ou texto) com os 10 pipelines, cada um identificado pela questão.
- Avaliação por critérios: estágios corretos, ordem, nomes de campos, resultado correto.

Checklist do Pipeline

1. FILTRE cedo ($match)
2. REDUZA campos ($project / $unset)
3. DESMONTE só se precisar ($unwind)
4. JUNTE ($lookup, $unionWith)
5. AGRUPE e CALCULE ($group, $cond, $switch)
6. ORDENE e LIMITE ($sort → $limit)
7. FORMATE a saída ($project, $round)

Boa sorte no teste! Documentação: mongodb.com/pt-br/docs/manual/aggregation/